



AWS Advance

A large blue circle on the left side of the slide, with a smaller purple circle at its bottom-left edge.

Today's Lesson

Load Balancer and Target Group
Cost optimizing and Cost calculation



Load Balancer and Target Group

Target Group

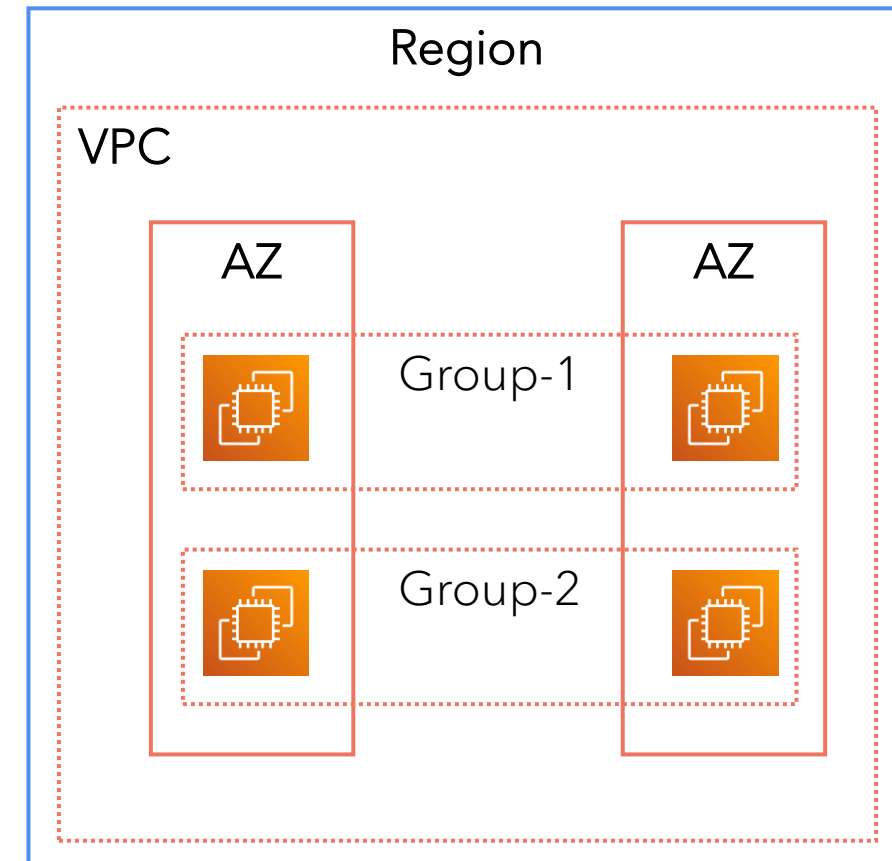
A **Target Group** is a logical grouping of *targets* (e.g., EC2 instances, IP addresses, Lambda functions, or containers running on ECS/EKS) that you register to receive traffic from a load balancer.

Targets Registration:

- You register one or more targets (EC2 instances, IPs, or Lambda functions) with a target group.
- Each target has a defined **port number** (for example, port 80 for HTTP).

Health Checks:

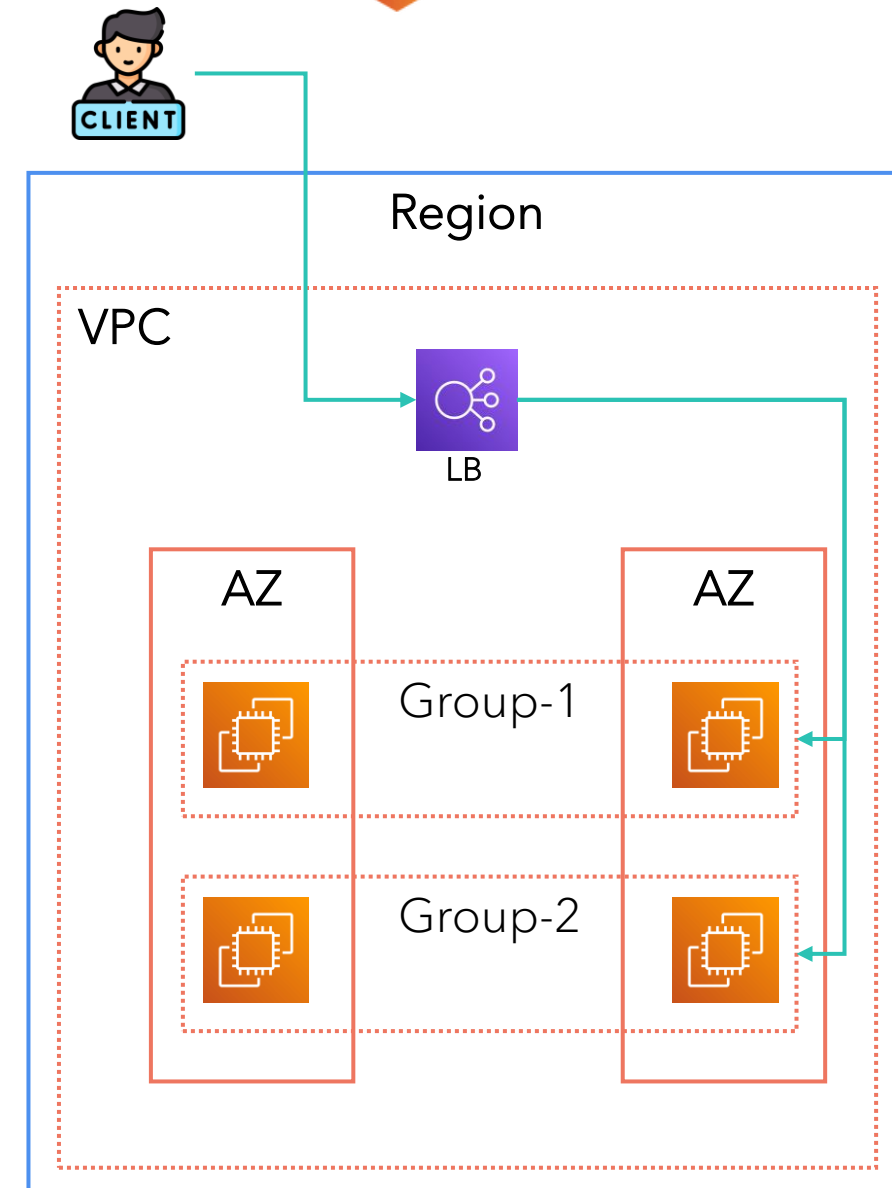
- The target group continuously performs **health checks** on registered targets.
- Only **healthy** targets receive traffic.



Load Balancer

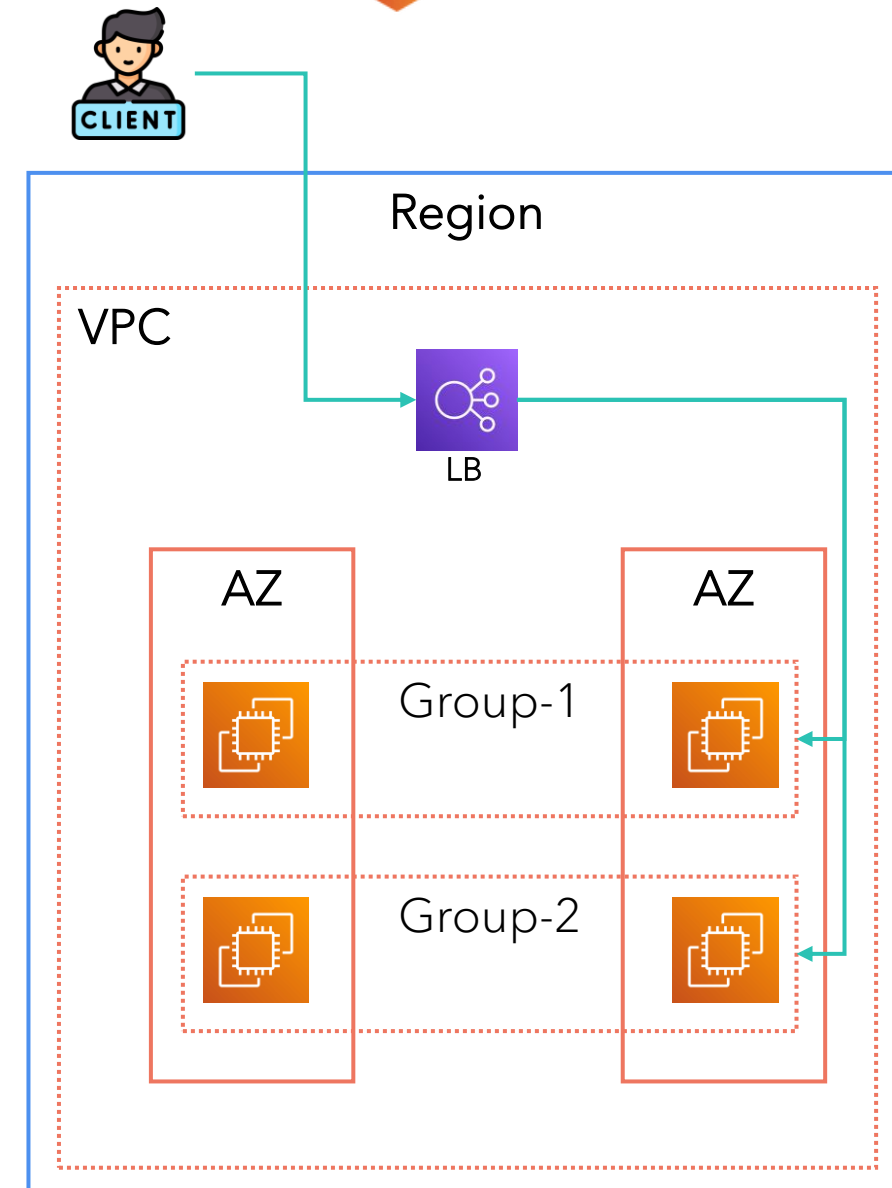
A **Load Balancer** acts as a single entry point for client requests and intelligently **distributes traffic** among **multiple backend resources** (called **targets**) to prevent any resource from being overloaded and to maintain performance and reliability.

For example, if you have a web app running on several EC2 instances, the load balancer sits between the clients (users) and these instances, distributing requests evenly among them.



Load Balancer

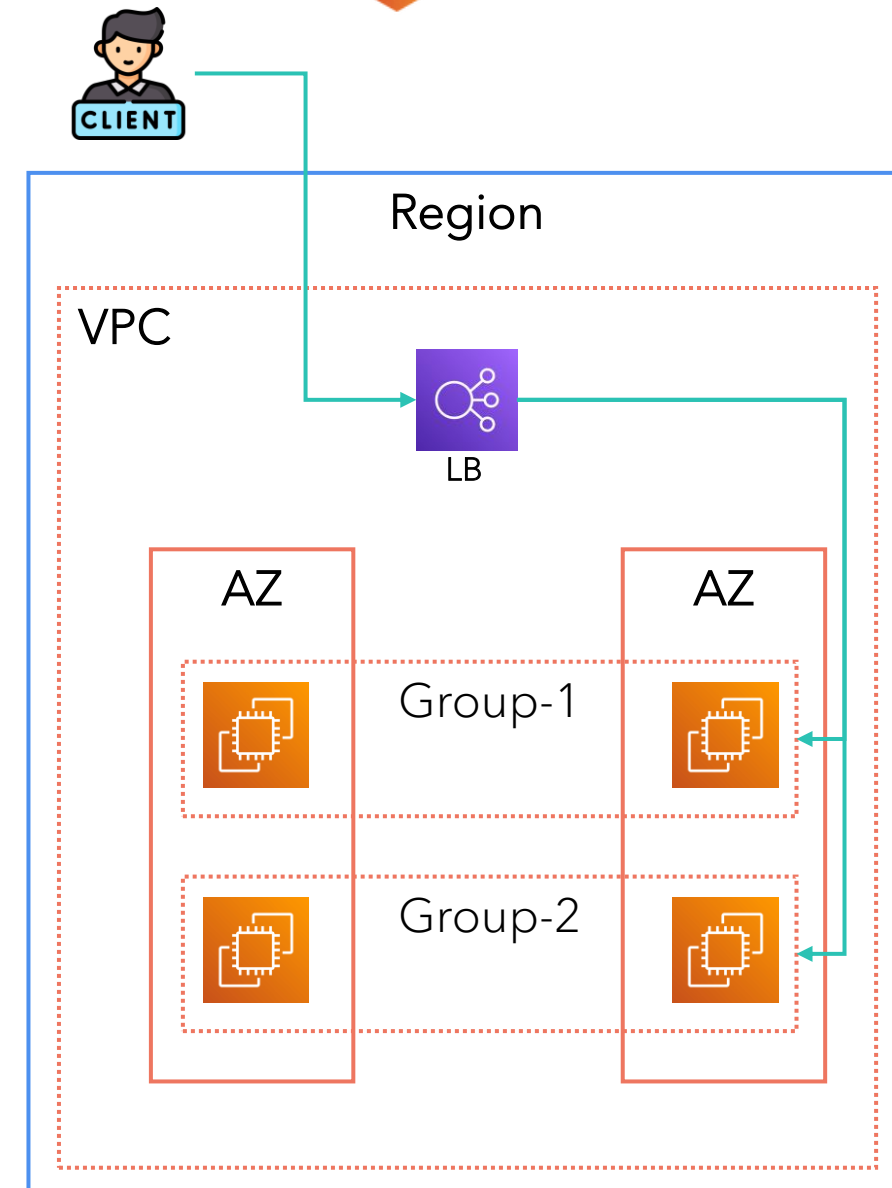
Load Balancer Type	Protocols Supported	Typical Use Case	Key Features
Application Load Balancer (ALB)	HTTP, HTTPS, WebSocket	Web and API-based apps	Layer 7 routing (content-based), host/path-based routing, integrates with ECS and Lambda
Network Load Balancer (NLB)	TCP, TLS, UDP	High-performance apps (e.g., gaming, real-time)	Ultra-low latency, handles millions of requests per second, static IP support
Gateway Load Balancer (GLB)	IP traffic	Security and network traffic inspection	Integrates third-party virtual appliances like firewalls or deep packet inspection systems



Load Balancer

Main Components

- **Listener** – Defines how the load balancer checks for connection requests.
(e.g., “Listen on port 80 for HTTP traffic.”)
- **Target Group** – Contains backend resources (EC2 instances, IPs, Lambda functions, containers) that receive traffic.
- **Health Checks** – Monitor the health of registered targets. Traffic is only sent to healthy targets.
- **Rules and Routing** – Determine how requests are routed
(e.g., path-based routing).



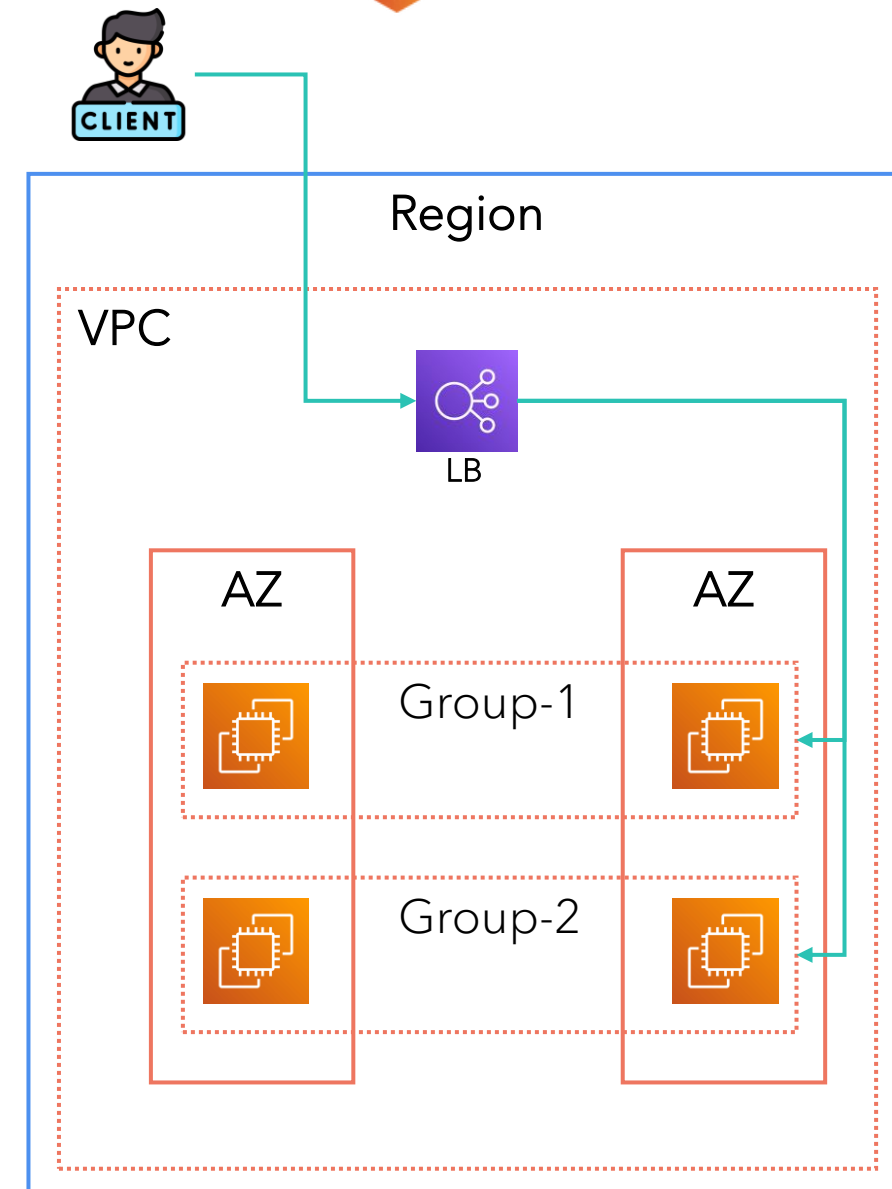
Load Balancer

How AWS Load Balancers Work

- Client sends a request → arrives at the Load Balancer.
- Load Balancer listener examines the request (based on port or protocol).
- Routing rules decide which *target group* should handle the request.
- The Load Balancer **forwards** that request to a healthy target.
- The **response** returns through the load balancer back to the client.

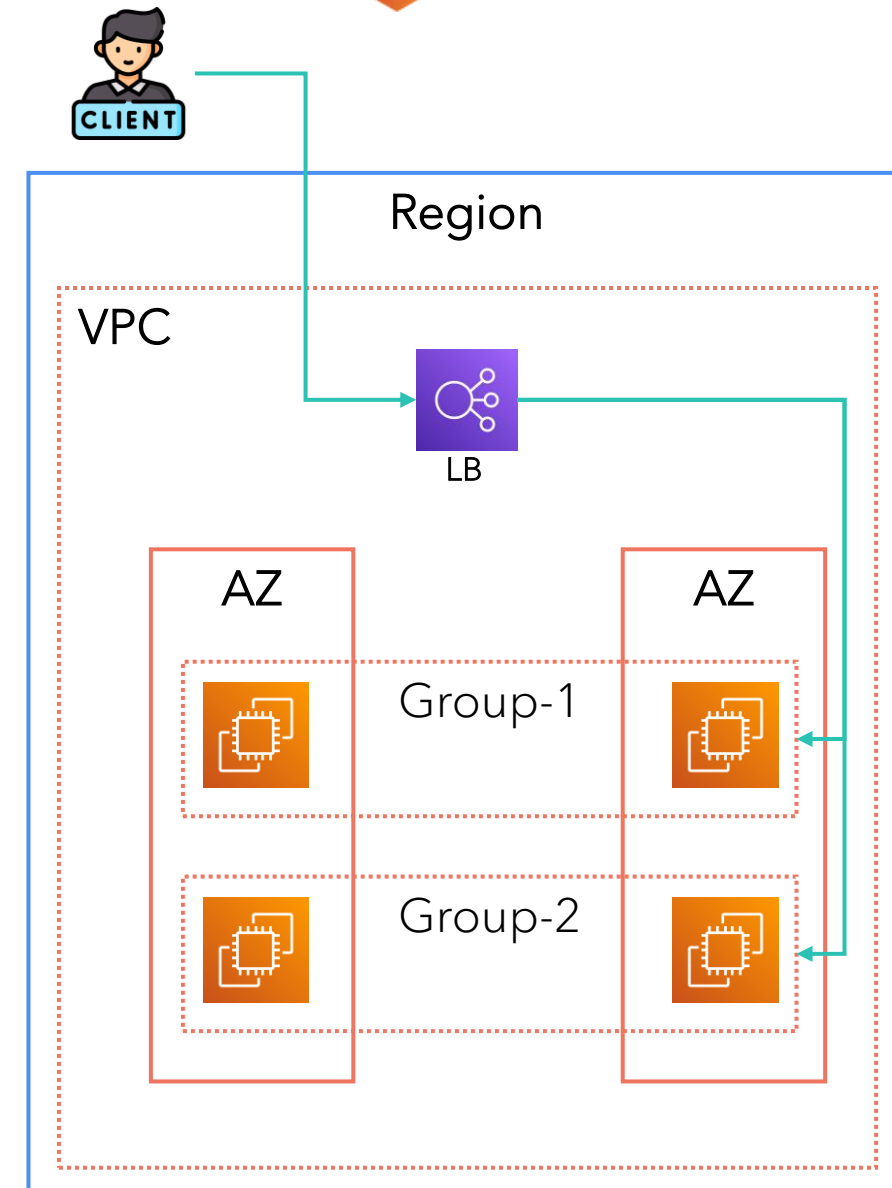
It ensures:

- Traffic is distributed evenly.
- Automatically removes unhealthy targets.
- Supports **cross-zone load balancing** (sends requests across multiple AZs).



Load Balancer

Load Balancer Type	Protocols Supported	Typical Use Case	Key Features
Application Load Balancer (ALB)	HTTP, HTTPS, WebSocket	Web and API-based apps	Layer 7 routing (content-based), host/path-based routing, integrates with ECS and Lambda
Network Load Balancer (NLB)	TCP, TLS, UDP	High-performance apps (e.g., gaming, real-time)	Ultra-low latency, handles millions of requests per second, static IP support
Gateway Load Balancer (GLB)	IP traffic	Security and network traffic inspection	Integrates third-party virtual appliances like firewalls or deep packet inspection systems





Cost Optimization & Estimation

Cost Optimization & Estimation

Why DevOps need to care about this?

- **Cost Estimation** means predicting how much your CI/CD pipelines, environments, applications, and infrastructure will cost *before* you deploy them.
- **Optimization** ensures that once resources are running, you're using them efficiently and not wasting money on over-provisioned or idle services.

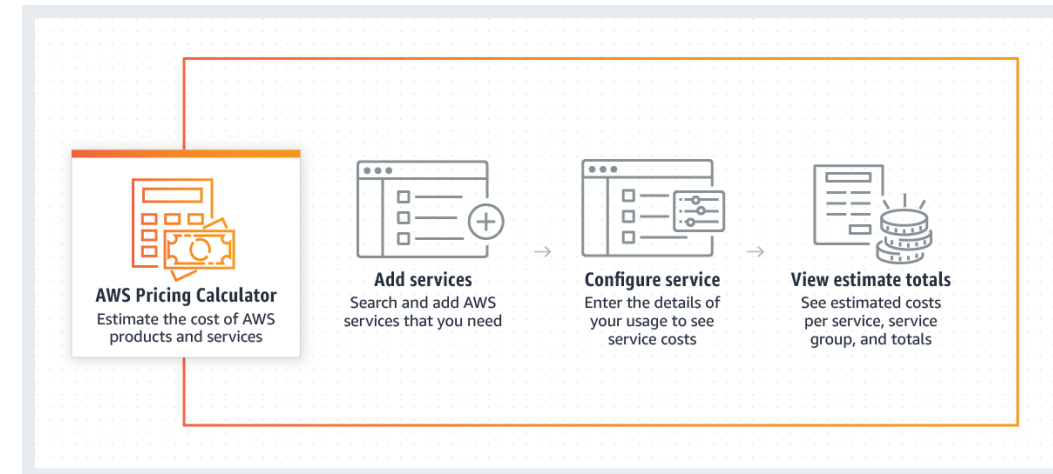
Benefit	Description
Predictable Bills	No more end-of-month billing shocks.
Improved Stakeholder Trust	Shows accountability for cloud spending.
Sustainable CI/CD Operations	Test and dev environments run efficiently.
Better Design Decisions	Encourages use of serverless or event-driven architectures.
Culture of Efficiency	Aligns with DevOps core philosophy – continuous improvement.

Cost Estimation

AWS cost estimation is the process of predicting how much your AWS infrastructure and services will cost – *before you deploy them*.

Since AWS uses a **pay-as-you-go** pricing model, cost estimation helps you:

- Plan budgets
- Prevent unexpected expenses
- Evaluate architectural cost trade-offs
- Optimize your resource usage



AWS Pricing Calculator

What it is: A web-based tool (previously called the Simple Monthly Calculator) that lets you estimate AWS service costs and compare various configurations.

URL: <https://calculator.aws>

Best for:

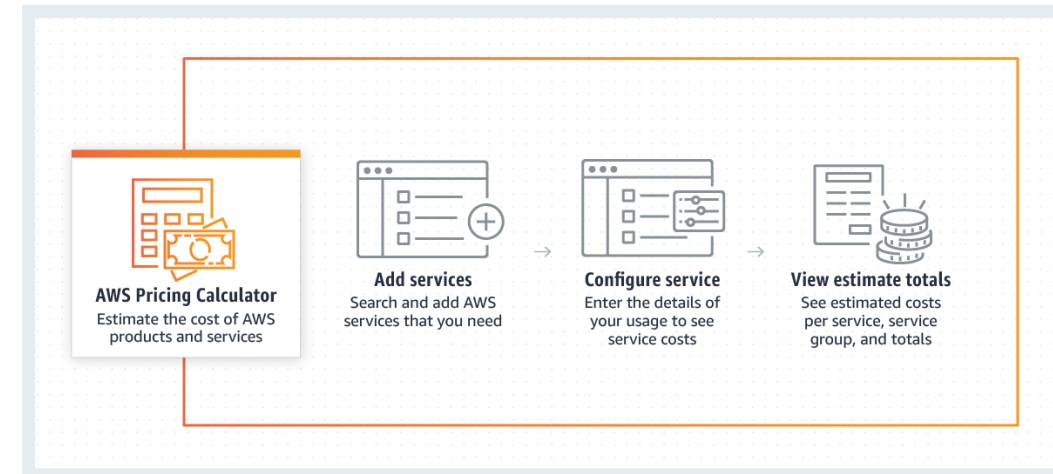
- Estimating costs before deployment
- Comparing pricing models or architectures

Example:

Estimate cost for:

- 2 EC2 instances (t3.medium) running 24x7 in US-East-1
- 100 GB Amazon S3 storage
- 20 GB monthly data transfer

The calculator shows expected monthly costs.



Cost Optimization

AWS Cost Optimization is the process of **reducing and controlling your AWS cloud spend** while ensuring your systems remain performant, scalable, and reliable.

It's part of the **AWS Well-Architected Framework**, under the **Cost Optimization Pillar**, which helps you design cost-efficient systems without sacrificing capability.

Some practices for optimizing cost:

- Right-Sizing Resources
- Use Appropriate Pricing Models
- Use Auto Scaling and Elastic Services



Right-Sizing Resources

"Avoid over-provisioning! "

- Analyze resource utilization (CPU, memory, IO) using **AWS CloudWatch**, **Compute Optimizer**, or **Trusted Advisor**.
- Resize or downgrade underutilized resources.

Examples:

- Change EC2 instance from **m5.large** (2 CPUs, 8GB) → **t3.medium** (2 CPUs, 4GB) if utilization <30%.
- Reduce EBS volumes capacity if consistently underused.

Use Appropriate Pricing Models

On-Demand Instances

- Pay-as-you-go – Good for unpredictable workloads.

Reserved Instances (RI)

- Commit to 1 or 3 years for up to **72% lower cost**.
- Best for predictable, steady workloads.

Savings Plans

- Flexible discount model similar to RIs but applies automatically across instance families and regions.

Spot Instances

- Use unused AWS capacity at up to **90% discount**, but can be interrupted anytime.
- Great for batch jobs, big data, containers, or CI/CD.

Use Auto Scaling and Elastic Services

"Scale up when needed; scale down automatically when not. "

- Auto Scaling Groups for EC2.
- **Lambda / Fargate** (serverless and container services) – pay only per request or compute time.
- Schedule scaling policies for predictable workloads (e.g., scale down nights/weekends).

Benefit: No paying for idle capacity.



Practice Lab